

DS6400 Portfolio

Stephen Olsen

Table of contents

Preface	3
I 01: Performance Measures	4
1 In class exploration and discussion	5
1.0.1 Exercise	5
1.0.2 Exercise	6
1.0.3 Exercise	7
1.0.4 Exercise [incomplete]	9
2 Week 1: Steyerberg et al. (2001)	10
2.1 Student exploration	10
3 Week 2: Efron 1997	24
3.1 Student exploration	24
3.1.1 Key Ideas	24
3.1.2 Simulation Plan	27
3.1.3 Definitions	28
3.1.4 Equations	30
3.1.5 Simulation	31
II 02: Visualizing the Model	41
4 In class exploration and discussion	42
5 Partial Dependence Plots (PDP)	43
5.1 Student Exploration	43
5.2 Motivating Idea for the PDP	43
5.3 Exploration	44

Preface

This is a Quarto book for capturing notes associated with DS6400 Advanced ML 1.

Original class notes are copied from <https://tgstewart.cloud/ml1/> to retain formulas and original exercises.

This is a book created from markdown and executable code. It contains topics on machine learning, estimation, and various mathematical concepts.

It will be continually populated over the course of the semester.

Part I

01: Performance Measures

1 In class exploration and discussion

1.0.1 Exercise

Consider two generic prediction models f_1 and f_2 for univariate, continuous outcome $Y \in \mathbb{R}$ with multivariate predictors $X \in \mathbb{R}^d$. Without knowing anything about how the prediction models were created, describe how you might evaluate the performance of f_1 and f_2 if you know the true relationship between X and Y . Try to be precise in your description, using mathematical notation if helpful.

In class we discussed X as a matrix D , for data, and Y as an output vector or single number P , for predictions. In between D and P there may be a D' matrix where the data is transformed. The prediction is related to some target quantity that we care about.

One way we might validate the model, is to run X through f to get $\hat{Y}$, and then we can compare $\hat{Y}$ to Y . The specific formula from class is below:

$$f : D \rightarrow D' \rightarrow P \quad f(x_0) = \hat{y}_0 \quad P \in \mathbb{R}$$

Then, we will have to consider what is the method of comparison between Y and $\hat{Y}$?

Some options include the following one-number summary values:

1. Mean Squared Error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

2. Root Mean Squared Error:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

3. Mean Absolute Error:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Alternatively, we might use a visualization to evaluate this. We would plot Y vs. $\hat{Y}$.

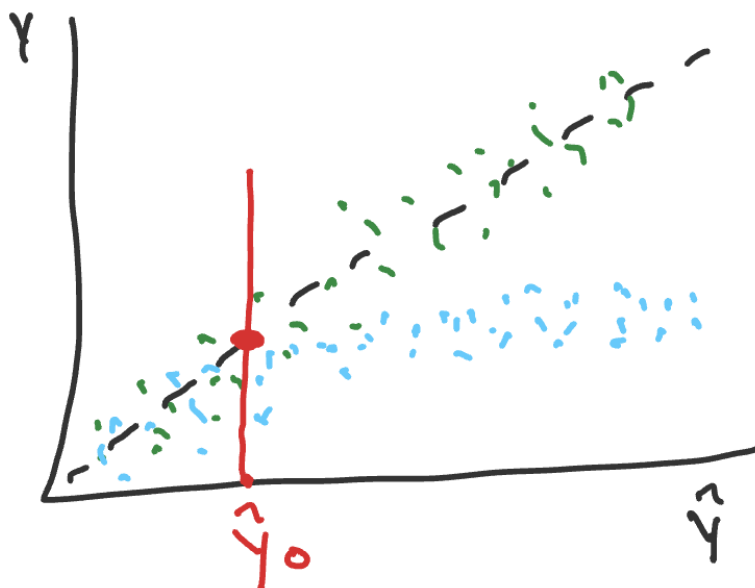


Figure 1.1: Calibration Curve

The plot above is the calibration curve. It is more descriptive than the available one number-summaries listed.

Q: What is the mathematical definition for “well-calibrated”? in the calibration curve? A: The expected value at y given a specific $\hat{y}_0$ is equal to $\hat{y}_0$.

$$\mathbb{E}[y \mid \hat{y}_0] = \hat{y}_0$$

OR

$$\mathbb{E}[y - \hat{y}_0 \mid \hat{y}_0] = 0$$

Calibration is all about the average or expectation, NOT about the spread around the line. Discrimination is another metric that looks at tightness around the line. These measures will be overly optimistic if we use the same data that built the model. More robust definitions and exploration of these concepts are continued below.

1.0.2 Exercise

Read [RMS 5.1.2](#). Describe/define/develop an example of the following terms

1. Discrimination (rank based and variation based)

Rank based discrimination is an assessment of whether predicted values are ordered correctly. From RMS 5.1.2, it is a rank correlation of $\hat{Y}$ and Y , with examples including Spearman's ρ , Kendall's τ , and Somers' D_{xy} . Variation based discrimination is an assessment of whether the predicted values differ or spread across predictions. From RMS 5.1.2, it is a measure based on variation in $\hat{Y}$, with examples including the regression sum of squares and the g -Index.

2. Calibration

Calibration is a measure of the average predictive accuracy for f . From RMS 5.1.2, these measures assess absolute prediction accuracy. *Calibration-in-the-large* compares the average $\hat{Y}$ with the average Y . Calibration curves are a part of the overall calibration measures definition.

3. Calibration curve

This is a plots Y values (actuals) versus $\hat{Y}$ values (predictions), which shows you the relationship between them, and can help assess performance at individual predicted values of $\hat{y}_0$. From RMS 5.1.2, a high-resolution calibration curve or *calibration-in-the-small* assesses the absolute forecast accuracy of predictions at individual levels of $\hat{Y}$. From class and as we can see below, this can help you observe calibration and discrimination.

From Claude's assistance: * Discrimination = "does it sort people correctly (and by how much)?" * Calibration = "are its actual numbers trustworthy?"

1.0.3 Exercise

1. Sketch a calibration curve that represents poor discrimination but good calibration

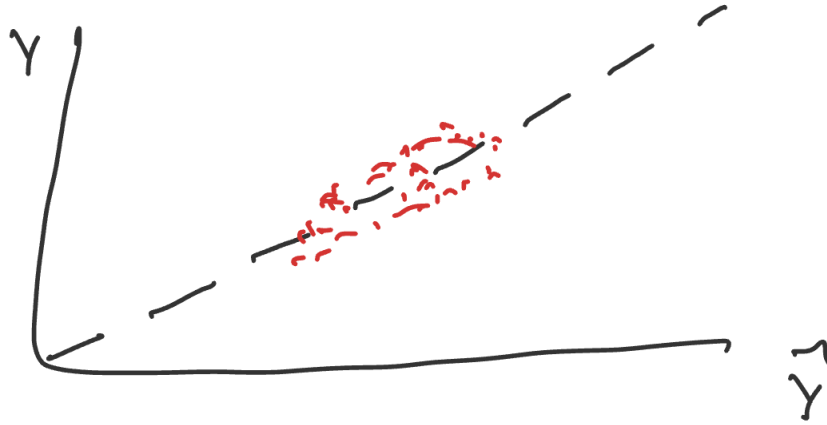


Figure 1.2: Good Calibration, Bad Discrimination

This plot is clustered around the line (good calibration), but it is not spread on $\hat{Y}$ (poor discrimination).

2. Sketch a calibration curve that represents good discrimination but poor calibration



Figure 1.3: Bad Calibration, Good Discrimination

This plot is spread on $\hat{Y}$ (good discrimination) but not following the line (poor calibration).

1.0.4 Exercise [incomplete]

```
# may need to be installed first pak::pak("thomasgstewart/ds6021b")  
require(ds6021b)
```

Consider the three predictor setting: $X \in \mathbb{R}^3$. Let

$$X_1, X_2 \sim N(0, 1) \quad X_3 \sim \text{BIN}(n = 1, p = 0.4)$$

$$Y \sim N(\mu(\mathbf{X}), 1) \quad \mu(\mathbf{X}) = \sqrt{5} \text{sigmoid}(X_1^2 + X_3) + \sqrt{5} \text{sigmoid}(X_2) X_3$$

1. Simulate data from the model.
2. Create the calibration curves for f_1 to f_5 , which are in the **ds6021b** package.
3. Calculate a global measure of model performance.
4. Add a loess curve to the calibration curve as a measure of local performance.

2 Week 1: Steyerberg et al. (2001)

2.1 Student exploration

1. Select one of the papers from the list below

Selected Paper: [Steyerberg et al. \(2001\)](#) - Internal validation of predictive models: efficiency of some procedures for logistic regression analysis

2. For the initial exploration, do the following:
 - Prepare a summary of the key ideas. Be prepared to describe/illustrate the concepts in a concrete example.
 - Review the instructions for the student presentation and come with an idea/sketch of the simulation you'll perform.

Key Ideas

1. From the opening line of the abstract: *“The performance of a predictive model is overestimated when simply determined on the sample of subjects that was used to construct the model.”* The authors expand on this idea of optimism. When the model is trained and evaluated on only data within the sample, the estimated performance will be optimistic compared with the real performance on new data that is completely outside the sample. This difference is between apparent performance and test performance.
2. From the closing paragraph of the introduction: *“In this study we compare the efficiency of internal validation procedures for predictive logistic regression models.”* (also basically the title lol) The authors walked through a comparison of the split-sample approach (test-train splits), cross-validation, and bootstrapping. Each one is an internal validation procedure that is commonly used for evaluating models.
3. From the opening line of section 2.4: *“We evaluated several internal validation procedures that aim to test performance more accurately than is achieved by the apparent performance estimate.”* The authors set out to determine which validation procedure of those listed above is best for the logistic regression model in question. They used the GUSTO-I dataset, detailed below, and fit multiple regression models for prediction.

Dataset: GUSTO-I; n=40,830; 2,851 deaths Working with a dataset of 30-day mortality after an acute myocardial infarction. What does that mean? They're looking at the % of patients who die from any cause within 30 days of a heart attack.

What they want us to do: Stop doing CV and split-sample (test-train splits), and start doing regular bootstrapping instead for estimation of accuracy!

How they showed it: Logistic regression was fitted in each sample with a previously specified set of eight dichotomous predictors (including shock, age>65, diabetes, etc). The apparent performance of the model fitted to the full GUSTO-I data was the reference point from which they compared the other performance measures. *"The large size of the total GUSTO-I data set makes sure that the optimism in this performance estimate is negligible."* As mentioned above, they compared variants of split-sample, cross-validation, and bootstrap (including the .632 and .632+ methods) specifically at EPV=10 to analyze estimates of model performance.

Some results:

- The optimism and the variability of the model performance estimates reduced as sample size increased.
- *"The regular bootstrap resulted in the most accurate estimates of model performance: the bias was close to zero and the variability similar to that of the apparent performance."*
- Bootstrap resampling resulted in stable and nearly unbiased estimates of performance.
- Relation to some other papers: *"a puzzling observation was that the .632 and the .632+ bootstrap variants were not superior to the regular bootstrap."*

Definitions

- *Events Per Variable (EPV):* The number of outcome events (i.e. number of deaths) divided by the number of predictor variables in the model. Recommended to be at least 10 to provide an adequate predictive model.
- *Internal Validation:* All methods of assessing performance using only the dataset you were provided.
- *External Validation:* Assessing performance using a TRULY NEW dataset. Something from another clinic, a separate experiment, real world data, etc.
- *Apparent Performance:* Predictive performance estimated on the training sample. Optimistically biased, because the same data is used to fit and evaluate the model.
- *Test Performance:* Predictive performance estimated on the test data (held-out sample), an independent test set consisting of all patients from GUSTO-I except the patients in the random sample.

- *Optimism*: The difference between apparent performance and a model's true performance on new data.
- *Split-sample approach*: Randomly split training data in two parts: one to develop the model and another to measure its performance.
- *Cross-validation*: Model is developed on one part of the data and tested on the other part, while these samples are randomly taken repeatedly and the average is taken as an estimate of the performance.
- *Bootstrap*: Replicates the process of sample generation from an underlying population by drawing samples with replacement from the original data set, of the same size as the original data set.
- *Discrimination*: The ability to distinguish high-risk subjects from low-risk subjects. Commonly quantified by a measure of concordance, the c statistic.
- *Calibration*: Whether the predicted probabilities agree with the observed probabilities. Well-calibrated models have a slope of 1, while models providing too extreme predictions have a slope less than 1. Well-calibrated models also have an intercept of 0, while models providing predictions that are too high or too low on average have an intercept different from 0.

Sketch of the simulation: Replicate a simulation from the paper

The goal is to demonstrate that the regular bootstrap returns the optimal estimates of internal validity.

- Start with the GUSTO-I dataset (from R or Python package)
 - Generate sample and independent test sets for $EPV = 10$
 - Complete split sample and regular bootstrap on the sample dataset
 - Compare Predictive performance (c statistic, calibration slope, Brier score, $D(=\text{scaled}\chi^2)$, R^2)
3. For the student presentation, do one of the following and come prepared to present it to the class:
- **Replicate a simulation or example from the paper** [SELECTION]
 - Develop your own simulation study which explores a concept discussed in the paper
 - Complete the simulation study described below.

Replicating the example from the paper

The following sections replicating the simulation were assisted by Claude. I am still working to develop a full understanding of the code, formulas, and methods completed within. Formulas and comments will be added progressively.

Dataset: GUSTO-I; $n=40,830$; 2,851 deaths Available in full anonymized version via `predtools` package in R.

Figures: Replicating a version of Figure 2 from the paper

In the paper, figure 2 shows bias and variability for a few of the procedures for estimating model performance in independent data for $EPV = 10$, including apparent performance, split-sample validation on 50% or 33% of the sample, cross-validation on 50% or 10% of the sample, and regular bootstrapping. In this simulation, I will show apparent performance, split-sample validation on 33% of the sample, and regular bootstrapping. I will also only show metrics for calibration and discrimination (c statistic, calibration slope, Brier score).

Methods: Replicating the steps of the paper

1. Load in the dataset
2. Define logistic regression model with 8 predictors from the paper
3. Generate a sub-sample for $EPV = 10$ (set a seed to keep results steady)
4. [Not needed for this simulation] Use model to compute the benchmark performance on the full dataset
5. Use model to compute apparent performance on sub-sample dataset [simulated many times to get bias and variability]
6. Use model to compute split-sample performance on sub-sample dataset [""]
7. Use model to compute bootstrap performance on sub-sample dataset [""]
8. Build figure 2 to compare performance!

```
## Loading in Gusto-I dataset from predtools
```

```
#install.packages("predtools")
library(predtools)
data(gusto)

str(gusto)
```

```
'data.frame':  40830 obs. of  29 variables:
 $ day30   : int  0 0 0 0 0 0 0 0 0 0 ...
 $ sho     : int  0 0 0 0 0 0 0 0 0 0 ...
 $ hig     : int  1 1 0 0 1 0 0 0 1 0 ...
 $ dia     : 'labelled' int  0 0 0 0 0 0 0 0 0 0 ...
 ..- attr(*, "label")= chr "Diabetes"
 $ hyp     : int  0 0 0 0 0 0 0 0 0 0 ...
 $ hrt     : int  0 1 0 0 1 1 1 0 0 1 ...
 $ ttr     : 'labelled' int  1 1 0 1 0 1 0 1 1 1 ...
 ..- attr(*, "label")= chr "Time to Relief of Chest Pain > 1h"
 $ sex     : Factor w/ 2 levels "male","female": 1 1 1 1 1 1 1 1 1 1 ...
 ..- attr(*, "label")= chr "Sex"
```

```

$ Killip : Factor w/ 4 levels "I","II","III",...: 1 2 1 1 1 1 1 1 1 1 ...
..- attr(*, "label")= chr "Killip Class"
$ age : num 19 19 20.3 20.8 21 ...
$ ste : 'labelled' int 4 6 5 8 5 1 0 4 8 6 ...
..- attr(*, "label")= chr "Number of ECG Leads with ST Elevation"
$ pulse : 'labelled' int 60 90 77 75 85 90 90 50 70 95 ...
..- attr(*, "label")= chr "Heart Rate"
..- attr(*, "units")= chr "beats/min"
$ sysbp : 'labelled' int 130 150 106 124 135 129 130 120 157 130 ...
..- attr(*, "label")= chr "Systolic Blood Pressure"
..- attr(*, "units")= chr "mmHg"
$ ant : int 1 1 0 0 1 0 0 0 1 0 ...
$ miloc : Factor w/ 3 levels "Inferior","Other",...: 3 3 1 1 3 1 2 1 3 1 ...
..- attr(*, "label")= chr "MI Location"
$ height : 'labelled' num 170 161 171 180 188 ...
..- attr(*, "label")= chr "Height"
..- attr(*, "units")= chr "cm"
$ weight : 'labelled' num 60 72 72.5 69 107 115 60 80 65 76 ...
..- attr(*, "label")= chr "Weight"
..- attr(*, "units")= chr "kg"
$ pmi : Factor w/ 2 levels "no","yes": 1 2 1 1 1 1 1 1 1 1 ...
..- attr(*, "label")= chr "Previous MI"
$ htn : 'labelled' int 1 2 1 1 1 1 1 1 1 1 ...
..- attr(*, "label")= chr "History of Hypertension"
$ smk : Factor w/ 3 levels "never","quit",...: 1 1 3 1 2 1 1 2 1 1 ...
..- attr(*, "label")= chr "Smoking"
$ pan : int 1 2 1 2 1 1 2 1 2 1 ...
$ fam : 'labelled' int 2 2 1 2 2 2 2 1 2 1 ...
..- attr(*, "label")= chr "Family History of MI"
$ prevcvd : int 1 1 1 1 1 1 1 1 1 1 ...
$ prevcabg: int 1 1 1 1 1 1 1 1 1 1 ...
$ regl : int 6 8 7 7 2 10 2 3 5 8 ...
$ grpl : int 17 24 18 19 6 30 5 9 15 24 ...
$ grps : int 98 53 40 42 95 67 13 19 32 54 ...
$ tpa : int 0 NA NA 1 0 1 NA NA 0 NA ...
$ tx : Factor w/ 3 levels "SK+tPA","SK",...: 2 1 1 3 2 3 1 1 2 1 ...
..- attr(*, "label")= Named chr "Tx in 3 groups"
.. ..- attr(*, "names")= chr "TX"

```

From the paper: “A logistic regression model was fitted in each sample consisting of a previously specified set of eight dichotomous predictors: shock, age>65 years, high risk (anterior infarct location or previous MI), diabetes, hypotension (systolic blood pressure <100 mmHg), tachycardia (pulse>80), relief of chest pain>1 hr, female

gender”

Variables we want are:

- Outcome: `day30`
- Predictors:
- Shock: `sho`
- Age > 65 years: `age` (needs transform to binary)
- High Risk: `ant` or `pmi` (needs transform to binary)
- Diabetes: `dia`
- Hypotension: `sysbp` (needs transform to binary)
- Tachycardia: `pulse` (needs transform to binary)
- Relief of chest pain > 1hr: `ttr`
- Female gender: `sex`

```
gusto_df <- data.frame(  
  day30 = as.integer(gusto$day30),  
  shock = as.integer(gusto$sho),  
  age65 = as.integer(gusto$age > 65),  
  highrisk = as.integer(gusto$ant == 1 | gusto$pmi == "yes"),  
  diabetes = as.integer(gusto$dia),  
  hypotension = as.integer(gusto$sysbp < 100),  
  tachycardia = as.integer(gusto$pulse > 80),  
  ttr = as.integer(gusto$ttr),  
  female = as.integer(gusto$sex == "female")  
)  
  
head(gusto_df)
```

	day30	shock	age65	highrisk	diabetes	hypotension	tachycardia	ttr	female
1	0	0	0	1	0	0	0	1	0
2	0	0	0	1	0	0	1	1	0
3	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	1	0
5	0	0	0	1	0	0	1	0	0
6	0	0	0	0	0	0	1	1	0

All variables are now transformed to match the “eight dichotomous predictors” from the paper. Below, the logistic regression model is defined in R code for the upcoming simulations.

```
model_formula <- day30 ~ shock + age65 + highrisk + diabetes + hypotension + tachycardia + t
```

From the paper: “For example, for EPV=10 and 7.0% mortality, all training and test samples consisted of 1145 and 39,685 patients, with 80 and 2771 deaths, respectively.”

```
## set seed for reproducibility
set.seed(17)

## Creating ID column for tracking which patients are in what samples
gusto_df$row_id <- seq_len(nrow(gusto_df))

## Function to draw the training sample for each time this needs to be run
draw_dev_sample <- function(data, sample_total = 1145, target_events = 80){

  ## EPV = 10 with 8 predictors requires 80 events (deaths)
  target_events <- target_events
  target_non_events <- sample_total - target_events

  ## Stratified sampling to ensure desired number of events
  # Split into events and non-events
  events_pool      <- gusto_df[gusto_df$day30 == 1, ]
  nonevents_pool   <- gusto_df[gusto_df$day30 == 0, ]

  # Draw the stratified subsample
  dev_events       <- events_pool[sample(nrow(events_pool), target_events), ]
  dev_nonevents    <- nonevents_pool[sample(nrow(nonevents_pool), target_non_events), ]

  dev_sample <- rbind(dev_events, dev_nonevents)

  # Shuffle row order so events aren't grouped together
  dev_sample <- dev_sample[sample(nrow(dev_sample)), ]
}

## Function to draw the testing sample (all the rows that are not in the training sample)
draw_test_set <- function(data, dev_sample) {
  data[!(data$row_id %in% dev_sample$row_id), ]
}
```

Building reusable functions for computing desired metrics:

Metric definitions

C-statistic (discrimination) — the probability that a randomly chosen patient who died is assigned a higher predicted risk than a randomly chosen patient who survived:

$$c = P(\hat{p}_i > \hat{p}_j \mid Y_i = 1, Y_j = 0)$$

Estimated empirically as the proportion of all (event, non-event) pairs correctly ordered by the model, with ties counted as half-correct:

$$\hat{c} = \frac{1}{n_1 n_0} \sum_{i: Y_i=1} \sum_{j: Y_j=0} [\mathbb{1}(\hat{p}_i > \hat{p}_j) + 0.5 \cdot \mathbb{1}(\hat{p}_i = \hat{p}_j)]$$

where n_1 is the number of events and n_0 is the number of non-events. In this case from the paper, since we have a binary outcome, c is identical to the area under the receiver operating characteristic (ROC) curve; c varies between 0.5 and 1.0 for sensible models (the higher the better).

Calibration slope — the coefficient β from an auxiliary logistic regression of the observed outcome on the linear predictor (logit of the model's predicted probability):

$$\text{logit}(P(Y = 1)) = \alpha + \beta \cdot \text{linear predictor}$$

A well-calibrated model has $\beta = 1$. $\beta < 1$ indicates predictions that are too extreme (overfitting); $\beta > 1$ indicates predictions that are too conservative.

Brier score — mean squared error between predicted probability and observed outcome:

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (\hat{p}_i - Y_i)^2$$

Ranges from 0 (perfect) to 0.25 (worthless) for a well-behaved binary outcome model. Exactly the same formula as MSE, but specifically used for binary prediction models.

```
library(pROC)
```

```
Type 'citation("pROC")' for a citation.
```

```
Attaching package: 'pROC'
```

```
The following objects are masked from 'package:stats':
```

```
cov, smooth, var
```

```

compute_metrics <- function(y, pred) {
  # Guard against predicted probabilities of exactly 0 or 1,
  # which would break the logit transform below
  eps <- 1e-10
  pred <- pmin(pmax(pred, eps), 1 - eps)

  # Discrimination: c-statistic (AUC)
  c_stat <- as.numeric(pROC::auc(y, pred, quiet = TRUE))

  # Calibration slope: regress outcome on the linear predictor (logit of pred)
  lp <- log(pred / (1 - pred))
  cal_fit <- glm(y ~ lp, family = binomial())
  cal_slope <- unname(coef(cal_fit)["lp"])

  # Brier score
  brier <- mean((y - pred)^2)

  c(c_stat = c_stat, cal_slope = cal_slope, brier = brier)
}

```

Computing apparent performance for the logistic regression model on the training sample dataset

```

get_apparent <- function(sample_data, formula) {
  fit <- glm(formula, data = sample_data, family = binomial())
  pred <- predict(fit, type = "response")
  list(fit = fit, metrics = compute_metrics(sample_data$day30, pred))
}

```

Defining the function to evaluate a given model on new data for the test performance

```

eval_on_data <- function(fit, newdata) {
  pred <- predict(fit, newdata = newdata, type = "response")
  compute_metrics(newdata$day30, pred)
}

```

Function for the split-sample validation, essentially running a test-train split on the development sample and fitting the model.

```

split_sample_validate <- function(sample_data, formula, train_frac = 0.6667) {
  n_train <- round(nrow(sample_data) * train_frac)
  train_idx <- sample(nrow(sample_data), n_train)

  train_set <- sample_data[train_idx, ]
  test_set <- sample_data[-train_idx, ]

  fit <- glm(formula, data = train_set, family = binomial())
  eval_on_data(fit, test_set)
}

```

Defining function for the regular bootstrap method. Setting $B=100$ as the authors did in the paper. Sampling from the sample distribution with replacement, then training on that sample. Computing performance in the following way: *The performance in the bootstrap sample represents estimation of the apparent performance, and the performance in the original sample represents test performance. The difference between these performances is an estimate of the optimism in the apparent performance. This difference is averaged to obtain a stable estimate of the optimism. The optimism is subtracted from the apparent performance to estimate the internally validated performance [7]: estimated performance = apparent performance – average(bootstrap performance – test performance).* In other words, we evaluate model_b^* on the bootstrap sample itself (its own “apparent” performance), and on the original sample (its “test” performance).

$$\text{optimism}_b = \text{Perf}(\text{model}_b^*, \text{sample}_b^*) - \text{Perf}(\text{model}_b^*, \text{sample}_{\text{orig}})$$

Average the optimism across all B replicates:

$$\overline{\text{optimism}} = \frac{1}{B} \sum_{b=1}^B \text{optimism}_b$$

Subtract this from the apparent performance of the model fitted on the original (development) sample to get the bias-corrected estimate:

$$\text{estimated performance} = \text{apparent performance} - \overline{\text{optimism}}$$

```

bootstrap_correct <- function(sample_data, formula, apparent_metrics, B = 100) {
  n <- nrow(sample_data)
  optimism_sum <- c(c_stat = 0, cal_slope = 0, brier = 0)

  for (b in 1:B) {

```

```

boot_idx <- sample(n, n, replace = TRUE)
boot_sample <- sample_data[boot_idx, ]

boot_fit <- glm(formula, data = boot_sample, family = binomial())

boot_apparent <- compute_metrics(
  boot_sample$day30,
  predict(boot_fit, type = "response")
)
boot_test <- compute_metrics(
  sample_data$day30,
  predict(boot_fit, newdata = sample_data, type = "response")
)

optimism_sum <- optimism_sum + (boot_apparent - boot_test)
}

avg_optimism <- optimism_sum / B
apparent_metrics - avg_optimism
}

```

Function 7: run one full repetition (ties everything together)

```

# This is the "outer loop body" -- one full pass of the pipeline,
# matching one box in each boxplot of Figures 1 and 2.
run_one_repetition <- function(data, formula, sample_total = 1145, target_events = 80, B = 1000) {

  dev_sample <- draw_dev_sample(data, sample_total, target_events)
  test_set <- draw_test_set(data, dev_sample)

  # Apparent performance + the fitted model (reused below)
  app <- get_apparent(dev_sample, formula)
  apparent_metrics <- app$metrics

  # Test performance: evaluate the dev_sample-fitted model on truly new patients
  test_metrics <- eval_on_data(app$fit, test_set)

  # Split-sample (50/50, matching your simplified scope)
  split_metrics <- split_sample_validate(dev_sample, formula, train_frac = 0.6667)

  # Bootstrap-corrected
  boot_metrics <- bootstrap_correct(dev_sample, formula, apparent_metrics, B = B)
}

```

```

# MATH NOTE: delta = method's estimate - test performance.
# delta = 0 means the method perfectly recovered true generalization
# performance for this repetition. This is exactly Figure 2's y-axis.
data.frame(
  method = rep(c("apparent", "split_sample", "bootstrap"), each = 3),
  metric = rep(c("c_stat", "cal_slope", "brier"), times = 3),
  delta = c(
    apparent_metrics - test_metrics,
    split_metrics - test_metrics,
    boot_metrics - test_metrics
  )
)
}

```

Run the simulation loop [decreased to 10 for faster preview]

```

# NOTE: reduced from the paper's 500 repetitions to keep runtime
# reasonable for a live presentation. State this explicitly when
# presenting -- it's a deliberate, documented scope reduction, not
# an error.
n_reps <- 10

set.seed(17)
results_list <- vector("list", n_reps)

for (i in seq_len(n_reps)) {
  results_list[[i]] <- run_one_repetition(
    data = gusto_df,
    formula = model_formula,
    sample_total = 1145,
    target_events = 80,
    B = 100
  )
}

results <- do.call(rbind, results_list)

```

Visualize: recreate Figure 2

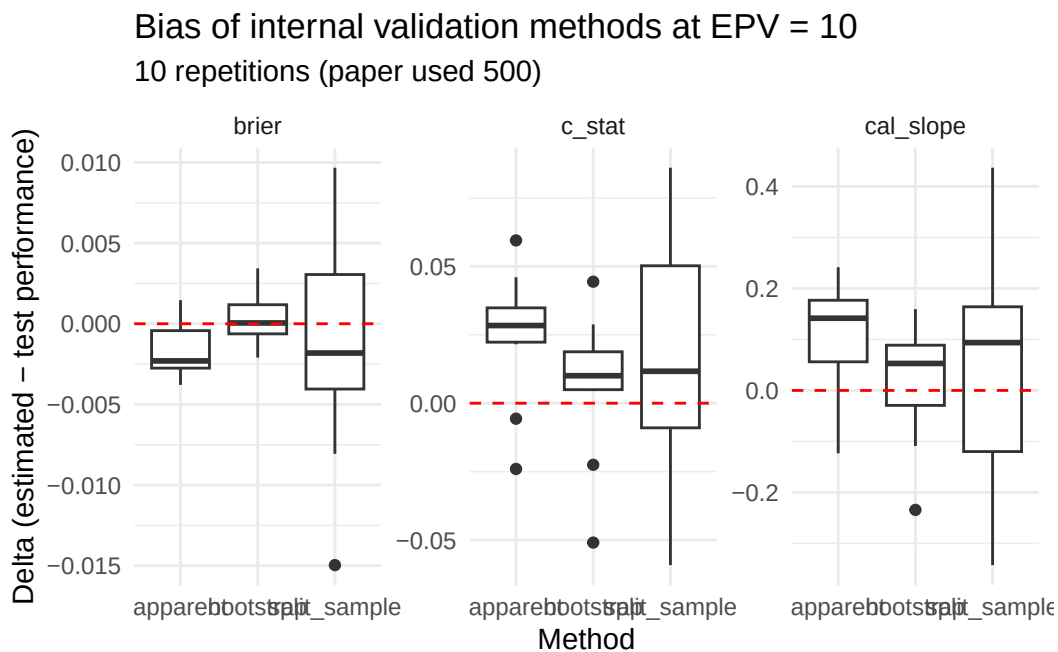
```

library(ggplot2)

ggplot(results, aes(x = method, y = delta)) +

```

```
geom_boxplot() +
geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
facet_wrap(~ metric, scales = "free_y") +
labs(
  title = "Bias of internal validation methods at EPV = 10",
  subtitle = paste0(n_reps, " repetitions (paper used 500)",
  x = "Method",
  y = "Delta (estimated - test performance)"
) +
theme_minimal()
```



Summarize bias and RMSE (reproduces Table 2's structure)

```
# MATH NOTE:
# Bias = mean(delta)                -- average signed error
# RMSE = sqrt(mean(delta^2))        -- root mean SQUARED error,
#                                   NOT centered around the bias.
# RMSE^2 = Bias^2 + Var(delta)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
summary_table <- results %>%
  group_by(method, metric) %>%
  summarise(
    bias = mean(delta),
    rmse = sqrt(mean(delta^2)),
    .groups = "drop"
  )
```

summary_table

```
# A tibble: 9 x 4
  method      metric      bias    rmse
  <chr>      <chr>    <dbl>  <dbl>
1 apparent  brier    -0.00173 0.00239
2 apparent  c_stat    0.0246  0.0335
3 apparent  cal_slope 0.102    0.147
4 bootstrap brier     0.000324 0.00154
5 bootstrap c_stat    0.00646  0.0260
6 bootstrap cal_slope 0.0135   0.111
7 split_sample brier    -0.00130 0.00681
8 split_sample c_stat    0.0169   0.0438
9 split_sample cal_slope 0.0630   0.245
```

3 Week 2: Efron 1997

3.1 Student exploration

Selected Paper: [Efron 1997](#) - Improvements on Cross-Validation: The .632+ Bootstrap Method

3.1.1 Key Ideas

3.1.1.1 Stop doing X, Start doing Y!

- Cross-validation is the standard way to estimate prediction error, but it's not good enough.
- Stop doing cross-validation, because the estimate of prediction error can be highly variable, even though it has low bias.
- Instead, use the bootstrap method with the .632+ rule, which performs better. Bootstrap procedures are smoothed versions of cross validation, with adjustments made to correct for bias.

3.1.1.2 Key Findings

- Of the 24 simulations completed, the results vary from experiment to experiment, but in terms of RMSE, the .632+ rule is an overall winner.
- The improvement in reducing variability of error rate predictions in the set of simulations is roughly equivalent to a 60% increase in the size of the training set.
- Bootstrap replications that give an estimate of prediction error can also be used to assess the variability of that estimate, which would be useful for inference, model selection, or comparing two models. Only studied on the leave-one-out bootstrap method.

3.1.1.3 Quick reminder

- What is cross validation?

- Repeated evaluation with partitions of a dataset used as test-train splits to calculate an average estimate of model performance and a sense of variance in performance across folds
- What is bootstrapping?
 - Replicates the process of sample generation from an underlying population by drawing samples with replacement from the original data set, of the same size as the original data set.
 - Used here for estimating variability / uncertainty of error rates
- How are they different?
 - Normal CV partitions the data into a fixed, small number of discrete splits (k-folds), where each data point is used for testing exactly once
 - Leave-one-out CV trains on exactly $n-1$ points and tests on the 1 left-out point, repeated once per observation
 - Bootstrapping resamples with replacement, generating a huge number of possible “pseudo-datasets” that vary continuously in composition, rather than a handful of fixed partitions
 - Leave-one-out bootstrap draws many bootstrap samples with replacement and, for each observation i , averages its error only over the bootstrap samples that happened to exclude i . Each observation’s error estimate is itself an average over many different training-set compositions
 - How is bootstrapping a “smooth” version?
 - * Because the bootstrap is averaged over many compositions, while the CV methods are dependent on cutoffs or possible variability in individual point predictions
- Why is this better?
 - Smoother estimates tend to have lower variance, which is especially useful for small datasets where a handful of CV folds can each look very different from one another

3.1.1.4 How they showed it

- Focused on the situation when the response is dichotomous (outcomes are 0, 1). Two of the explored prediction rules were Fisher’s linear discriminant function (LDF), and the nearest-neighbor (NN) rule. Examples of these are shown in figure 1.

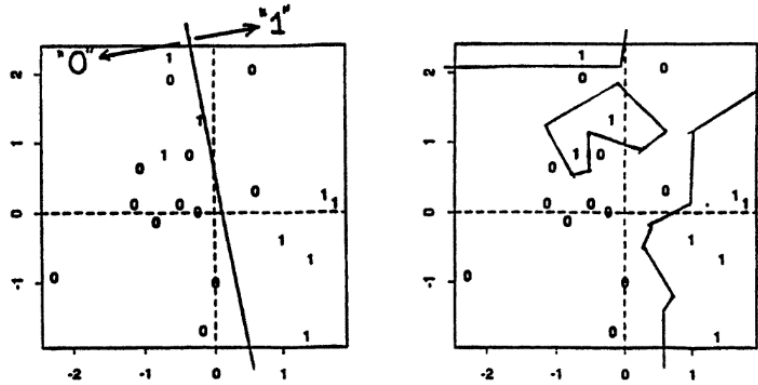


Figure 1. A Training Set Consisting of $n = 20$ Observations, 12 Labelled 0 and 8 Labelled 1. (a) The linear discriminant function predicts 0 to lower left of the solid line and 1 to upper right; (b) the nearest-neighbor rule predicts 1 in the three indicated islands and 0 elsewhere.

Figure 3.1: Figure 1. Showing examples of LDF and NN.

- Showed cross-validation as a nearly unbiased estimator of true error rate for both LDF and NN, but bootstrap-based estimator has RMSE 80% as large. Table 1 shows these simulation results.

Table 1. Error Rate Estimation for Situation (1)

	LDF		NN	
	Exp	RMS	Exp	RMS
CV1	.362	.123	.419	.123
.632+	.357	.096	.380	.099
True	.357		.418	

NOTE: CV1 is the cross-validation estimate based on omitting one observation at a time from the training set; 632+ is the bootstrap-based estimator described in Section 3. The table shows the expectation and RMS error of the two estimates for both the LDF and NN prediction rules. In both cases, $\text{RMS}(.632+)/\text{RMS}(\text{CV1})$ is about 80%.

Figure 3.2: Table 1. Showing comparison of CV and Bootstrap estimators.

- While the authors only studied classification problems with 0-1 loss, they claim that the results are non parametric, so they can be applied to any prediction rule. However, regression problems may have different behavior and the statistical approach may differ.
- Derived the Leave-one-out bootstrap estimator, which is a smoothed version of the leave-one-out cross-validation estimator. This is used in conjunction with the apparent

error rate to create the .632 estimator, and in turn, the .632+ rule. Further explanation in the Equations section below.

$$\widehat{\text{Err}}^{(.632+)} = \widehat{\text{Err}}^{(.632)} + \left(\widehat{\text{Err}}^{(1)'} - \overline{\text{err}} \right) \cdot \frac{0.368 \cdot 0.632 \cdot \hat{R}'}{1 - 0.368 \hat{R}'}$$

- Performed 24 sampling experiments, to investigate the effects of different classifiers, training set sizes, signal-to-noise ration, number of classes, and number of observations.
- The results vary considerably from experiment to experiment, but in terms of RMSE, the .632+ rule is an overall winner.

3.1.2 Simulation Plan

Complete the simulation study outlined on the class notes page.

3.1.2.1 Simulation study (Efron and Tibshirani 1997)

The goal of Efron and Tibshirani 1997 was to persuade readers to use the 0.632+ bootstrap estimate of the error rate rather than alternatives. To demonstrate its superiority, the authors began with a simple simulation study involving a binary outcome (Y_i) and two predictors ($\mathbf{T}_i = (T_{i1}, T_{i2})$). The simulation study was to demonstrate that the variance of the estimator (and consequently the mean squared error) of the estimator was lower.

Data generation model

$$Y_i \sim \text{BIN}(1, 0.5)$$

$$\mathbf{T}_i | Y_i = y_i \sim N_2 \left(\begin{bmatrix} y_i - 1/2 \\ 0 \end{bmatrix}, I_2 \right)$$

where I is the identity matrix and N_2 is the bivariate normal distribution.

Methods evaluated

1. Leave-one-out cross-validation (LOO CV)
2. 0.632+ bootstrap
3. Bootstrap

Models evaluated

1. LDA

2. 1-NN
3. Regression with polynomial surface (logistic)

Simulation parameters

1. $N = 10, 20, 30, 40, 50, 75, 100$

Plan

1. Before beginning the simulation, consider how you will display the results. Devise a figure that will succinctly show the results and support the goal of the paper.
2. Consider modular programming
3. Start with a single simulation setting (method/model/N).

3.1.3 Definitions

- Cross Validation
 - Repeated evaluation with partitions of a dataset used as test-train splits to calculate an average estimate of model performance and a sense of variance in performance across folds
- Variance
 - Model is too sensitive to training data, so it captures noise too. Poor prediction on unseen new data.
 - High variance = model is overfit
 - Low variance = model is more stable
 - Variance = $\mathbb{E} \left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2 \right]$
 - * Where,
 - * $\hat{f}(x)$: predicted value by the model
 - * $\mathbb{E}[\hat{f}(x)]$: average prediction over multiple training sets
- Bias
 - Error that occurs when a model is too simple to capture the true patterns in the data.
 - High bias = model is underfit
 - Low bias = model is better fit to patterns
 - Bias² = $(\mathbb{E}[\hat{f}(x)] - f(x))^2$
 - * Where,
 - * $\hat{f}(x)$: predicted value by the model
 - * $f(x)$: true value
 - * $\mathbb{E}[\hat{f}(x)]$: expected prediction over different training sets

- Error Rate
 - Apparent Error (rate): error rate calculated when evaluating the model on the data it was trained on, typically underestimates true error (optimistic)
 - * also referred to as training error
 - True Error (rate): error rate calculated when evaluating the model on new / unseen data
 - * also referred to as generalization error
 - Test Error (rate): error rate calculated on a held out test set, used to estimate true error
 - No-Information Error (rate): error rate that would occur if predictors had no relationship to the response
 - * simulating what error looks like under pure guessing (section 3)
 - * used by .632+ estimator to determine how much to trust the apparent error and bootstrap error
- Relative Overfitting Rate ($\hat{R}$)
 - a measure of how much a model is overfitting
 - scaled between 0 (no overfitting) and 1 (overfitting equal to the no-information value)
- Mean Squared Error (MSE)
 - Average of the squared differences between predicted values and actual values
- Prediction error
 - difference between predicted value and actual value (residual)
 - aggregated across many residuals using MSE, RMSE, or other statistics.
- Bootstrap
 - Replicates the process of sample generation from an underlying population by drawing samples with replacement from the original data set, of the same size as the original data set.
 - used here for estimating variability / uncertainty of error rates
- Point Estimate
 - A single value calculated from sample data used as the best guess for an unknown population parameter
 - Example: a computed error rate, such as apparent error, as a point estimate of the true error rate
- Parametric

- Fixed number of parameters estimated from the training data and used to make predictions
- Examples: Linear regression, logistic regression
- Nonparametric
 - Make few assumptions about functional form and relationships between variables
 - number of parameters grows with the size of the data and are flexible for modeling complex relationships
 - Example: k-NN, Decision Trees
- 0-1 loss
 - assignment of a 0 for a correct classification and a 1 for an incorrect classification
 - also called “indicator loss”
- Fisher’s linear discriminant
 - classification method used as a prediction rule for separating data into classes or groups based on a linear boundary
 - smooth, because there is a single line dividing groups
- Nearest neighbors
 - classification method used for separating data into classes or groups based on the closest points
 - unsmooth, because the decision boundary is local to closest points

3.1.4 Equations

Equation 8: leave-one-out cross-validation estimate of prediction error

$$\widehat{\text{Err}}^{(cv1)} = \frac{1}{n} \sum_{i=1}^n Q[y_i, r_{\mathbf{x}_{(i)}}(\mathbf{t}_i)] = \frac{1}{n} \sum_{i=1}^n Q(x_i, \mathbf{x}_{(i)})$$

where $\mathbf{x}_{(i)}$ is the training set with the i th observation removed.

$Q[y, r]$ is the discrepancy between a predicted value r and the actual response y .

Each observation is $x_i = (\mathbf{t}_i, y_i)$, where $\mathbf{t}_i$ is the predictor/feature vector and y_i is the response.

$r_{\mathbf{x}}(\mathbf{t})$ denotes the prediction rule, fit on training set $\mathbf{x}$, evaluated at feature vector $\mathbf{t}$.

Equation 14: leave-one-out bootstrap estimator (smoothed version of cross-validation)

$$\widehat{\text{Err}}^{(1)} = \frac{1}{n} \sum_{i=1}^n E_{\hat{F}_{(i)}} \{Q(x_i, \mathbf{x}^*)\}$$

where $\mathbf{x}^*$ is a bootstrap sample drawn from $\hat{F}_{(i)}$, the empirical distribution on $\mathbf{x}_{(i)}$.

Equation 24: The equation for the .632 estimator from Efron (1983):

$$\widehat{\text{Err}}^{(.632)} = 0.368 \cdot \overline{\text{err}} + 0.632 \cdot \widehat{\text{Err}}^{(1)}$$

This estimator is designed to correct upward bias in $\widehat{\text{Err}}^{(1)}$ by averaging it with the downwardly biased estimate $\widehat{\text{Err}}^{(.632)}$.

The coefficients $.368 = e^{-1}$ and $.632$ were suggested by an argument based on the fact that bootstrap samples are supported on approximately $.632n$ of the original data points.

The new equation for the .632+ rule, introduced by this paper, puts greater weight on $\widehat{\text{Err}}^{(1)}$ in situations where the amount of overfitting $\widehat{\text{Err}}^{(1)} - \overline{\text{err}}$ is large.

Equation 32: This equation is used in the simulation experiments of section 4.

$$\widehat{\text{Err}}^{(.632+)} = \widehat{\text{Err}}^{(.632)} + \left(\widehat{\text{Err}}^{(1)'} - \overline{\text{err}} \right) \cdot \frac{0.368 \cdot 0.632 \cdot \hat{R}'}{1 - 0.368 \hat{R}'}$$

Equation 28: The relative overfitting rate

$$\hat{R} = \frac{\widehat{\text{Err}}^{(1)} - \overline{\text{err}}}{\hat{\gamma} - \overline{\text{err}}}$$

$\hat{R}$ ranges from 0, with no overfitting ($\widehat{\text{Err}}^{(1)} = \overline{\text{err}}$), to 1, with overfitting equal to the no-information value $\hat{\gamma} - \overline{\text{err}}$.

3.1.5 Simulation

What actually took place: - Generated data according to them model - Implemented functions for running simulation with LDF Classifier - Replicated resultts from Table 3, Experiment 3 for the LDF Classifier

1. Data generation model

$$Y_i \sim \text{BIN}(1, 0.5)$$

$$\mathbf{T}_i | Y_i = y_i \sim N_2 \left(\begin{bmatrix} y_i - 1/2 \\ 0 \end{bmatrix}, I_2 \right)$$

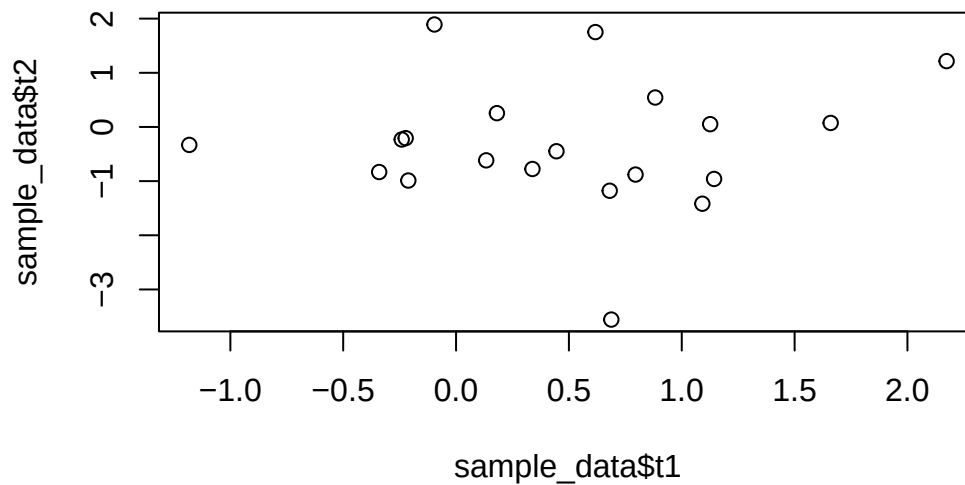
where I is the identity matrix and N_2 is the bivariate normal distribution.

```

## Data generating function
## n: number of observations to simulate
## Returns a data frame with columns y, t1, t2 matching Equation (1)
generate_data <- function(n) {
  y <- rbinom(n, size = 1, prob = 0.5)
  t1 <- rnorm(n, mean = y - 0.5, sd = 1)
  t2 <- rnorm(n, mean = 0, sd = 1)
  data.frame(y = y, t1 = t1, t2 = t2)
}

## Quick sanity check
set.seed(17)
sample_data <- generate_data(20)
#table(sample_data$y)
#head(sample_data)
plot(sample_data$t1, sample_data$t2)

```



2. Set up supporting functions

- Q Loss Function
- LDF Model
- True Error Function
- LOO-CV Function

```

## Equation 2  $Q[y, r]$ 
## y is the response, r is the predicted value
## Vectorized: y and r can be vectors of the same length,
## returning a vector of 0/1 losses (mean() of this gives an error rate)
qloss_function <- function(y, r) {
  ifelse(r == y, 0, 1)
}

```

```

# LDA function comes from MASS library
library(MASS)

## Fit LDF on a training set and predict on given data
ldf_fit <- function(train_data) {
  lda(y ~ t1 + t2, data = train_data)
}

## Returns numeric 0/1 predictions (not the raw factor from predict())
ldf_predict <- function(fit, newdata) {
  pred <- predict(fit, newdata = newdata)
  as.numeric(as.character(pred$class))
}

## Sanity check: apparent error rate for one training sample (n = 20)
set.seed(17)
sample_data <- generate_data(20)
ldf_model <- ldf_fit(sample_data)
preds <- ldf_predict(ldf_model, sample_data)

apparent_error <- mean(qloss_function(sample_data$y, preds))
cat(sprintf("Apparent error for LDF model with n = %d observations: %.3f\n",
  nrow(sample_data), apparent_error))

```

Apparent error for LDF model with n = 20 observations: 0.350

```

## Equation 5: true error rate for a fitted model
## fit: a fitted model object (e.g., from ldf_fit())
## predict_fn: the matching predict function (e.g., ldf_predict())
## n_test: size of the large independent test set used to
## approximate  $Err(x, F)$  -- the bigger n_test, the less
## noise in this estimate itself
true_error <- function(fit, predict_fn, n_test = 10000) {

```

```

test_data <- generate_data(n_test)
preds <- predict_fn(fit, test_data)
mean(qloss_function(test_data$y, preds))
}

## Sanity check: true error for the LDF model fit earlier (n = 20 training sample)
true_err <- true_error(ldf_model, ldf_predict)
cat(sprintf("True error for LDF model (n_test = %d): %.3f\n", 10000, true_err))

```

True error for LDF model (n_test = 10000): 0.319

```

## Equation 8: leave-one-out cross-validation estimate of prediction error
## data:      the full training sample (data frame with columns y, t1, t2)
## fit_fn:    model fitting function (e.g., ldf_fit())
## predict_fn: matching predict function (e.g., ldf_predict())
##
## For each observation i: fit on all OTHER n-1 points (x_(i)),
## predict on the held-out point i, and score that one prediction.
## Averaging these n held-out losses gives Err^(cv1).
loo_cv_error <- function(data, fit_fn, predict_fn) {
  n <- nrow(data)
  losses <- numeric(n)

  for (i in seq_len(n)) {
    train_data <- data[-i, ]
    test_point <- data[i, , drop = FALSE]

    fit <- fit_fn(train_data)
    pred <- predict_fn(fit, test_point)

    losses[i] <- qloss_function(test_point$y, pred)
  }

  mean(losses)
}

## Sanity check: LOO-CV estimate for the same n = 20 sample used above
loo_cv_err <- loo_cv_error(sample_data, ldf_fit, ldf_predict)
cat(sprintf("LOO-CV error for LDF model (n = %d): %.3f\n",
            nrow(sample_data), loo_cv_err))

```

LOO-CV error for LDF model (n = 20): 0.450

```

## First define bootstrap function
## Includes note to save the index of the sample data, so that the
## leave-one-out bootstrap can utilize that information
##
## NOTE: redraws if the resample happens to contain only one class --
## rare, but possible for small/imbalanced n, and LDA cannot fit
## without both classes present. This is a small, standard guard
## against a degenerate resample, not a change to the bootstrap logic.
draw_bootstrap_sample <- function(data) {
  repeat {
    idx <- sample(nrow(data), nrow(data), replace = TRUE)
    if (length(unique(data$y[idx])) > 1) break
  }
  list(sample_data = data[idx, ], included_idx = idx)
}

```

```

## Equations 14-17: leave-one-out bootstrap estimate  $Err^{(1)}$ 
## data:      the full training sample
## fit_fn:    model fitting function (e.g., ldf_fit())
## predict_fn: matching predict function (e.g., ldf_predict())
## B:        number of bootstrap replications (paper uses  $B = 50$ )
##
## For each of the  $B$  bootstrap resamples: fit on the resample, then
## predict on ALL  $n$  original points at once. For each original point  $i$ ,
## keep that prediction's loss ONLY if  $i$  happened to be excluded from
## this particular resample (Equation 15's  $I_i^b$ ). Averaging point  $i$ 's
## kept losses over just those "excluded" replicates gives  $E_i$ 
## (Equation 17); averaging  $E_i$  over all  $i$  gives  $Err^{(1)}$  (Equation 14).
loo_bootstrap_error <- function(data, fit_fn, predict_fn, B = 50) {
  n <- nrow(data)

  ## Running totals per original observation  $i$ :
  ##   loss_sum[i] = sum of  $Q_i^b$  over replicates  $b$  where  $i$  was excluded
  ##   exclude_n[i] = number of replicates where  $i$  was excluded
  loss_sum <- numeric(n)
  exclude_n <- numeric(n)

  for (b in seq_len(B)) {
    boot <- draw_bootstrap_sample(data)
    fit <- fit_fn(boot$sample_data)

    ## Predict on every original point at once -- we need  $Q_i^b$  for

```

```

## all i, then we'll discard the ones where i WASN'T excluded.
preds <- predict_fn(fit, data)
losses <- qloss_function(data$y, preds)

## TRUE for observations NOT drawn into this bootstrap resample
excluded <- !(seq_len(n) %in% boot$included_idx)

loss_sum[excluded] <- loss_sum[excluded] + losses[excluded]
exclude_n[excluded] <- exclude_n[excluded] + 1
}

E_i <- loss_sum / exclude_n
mean(E_i)
}

## Sanity check: leave-one-out bootstrap error for the same n = 20 sample
set.seed(17)
loo_boot_err <- loo_bootstrap_error(sample_data, ldf_fit, ldf_predict, B = 50)
cat(sprintf("Leave-one-out bootstrap error for LDF model (n = %d, B = %d): %.3f\n",
           nrow(sample_data), 50, loo_boot_err))

```

Leave-one-out bootstrap error for LDF model (n = 20, B = 50): 0.450

```

## Equation 24: the .632 estimator
## apparent_error: err-bar, the resubstitution/training error
## err1: Err^(1), the leave-one-out bootstrap error
estimator_632 <- function(apparent_error, err1){
  (.368*apparent_error) + (.632*err1)
}

## Sanity check: .632 estimate for the same n = 20 sample, using the
## apparent_error and loo_boot_err already computed above
err_632 <- estimator_632(apparent_error, loo_boot_err)
cat(sprintf(".632 error for LDF model (n = %d): %.3f\n",
           nrow(sample_data), err_632))

```

.632 error for LDF model (n = 20): 0.413

```

## Equations 27-32: the .632+ estimator

## Equation 27: No information error rate (gamma-hat)

```

```

## ps: observed proportion of y == 1 (p-hat-1)
## qs: observed proportion of predictions == 1 (q-hat-1)
gamma_hat <- function(ps, qs){
  (ps*(1-qs)) + ((1-ps)*qs)
}

## Equation 31: Relative overfitting rate
## NOTE: uses the raw err1 (not the truncated err1'), per the paper's definition
## NOTE: the paper describes R-hat as ranging from 0 to 1, but the raw
## formula has no upper cap -- if gamma_hat lands only slightly above
## apparent_error, the ratio can explode (confirmed via simulation: some
## replicates produced R_hat_prime in the thousands/billions). Capping
## at 1 enforces the paper's stated range and avoids a near-zero or
## negative denominator later in the .632+ formula.
R_hat_prime <- function(err1, apparent_error, gamma_hat_val){

  cond <- (err1>apparent_error) && (gamma_hat_val>apparent_error)
  raw <- ifelse(cond, (err1-apparent_error)/(gamma_hat_val-apparent_error), 0)
  #pmin(raw, 1)

}

## Equation 32: .632+ Rule used for simulations
estimator_632_plus <- function(err_632, err1, gamma_hat_val, apparent_error, r_hat_prime_val)

  ## Equation 31's truncated leave-one-out bootstrap error, Err^(1)'
  err1prime <- pmin(err1, gamma_hat_val)

  ## Calculate estimator
  numerator <- (.368*.632*r_hat_prime_val)
  denominator <- (1-(.368*r_hat_prime_val))

  err_632+((err1prime - apparent_error)*(numerator/denominator))
}

## Sanity check: .632+ estimate for the same n = 20 sample
set.seed(17)
sample_data <- generate_data(20)
ldf_model <- ldf_fit(sample_data)
preds <- ldf_predict(ldf_model, sample_data)
apparent_error <- mean(qloss_function(sample_data$y, preds))

```

```

loo_boot_err <- loo_bootstrap_error(sample_data, ldf_fit, ldf_predict, B = 50)
err_632 <- estimator_632(apparent_error, loo_boot_err)
gamhat <- gamma_hat(mean(sample_data$y), mean(preds))
rhatprime <- R_hat_prime(loo_boot_err, apparent_error, gamhat)
err_632_plus <- estimator_632_plus(err_632, loo_boot_err, gamhat, apparent_error, rhatprime)
cat(sprintf(".632+ error for LDF model (n = %d): %.3f\n",
            nrow(sample_data), err_632_plus))

```

.632+ error for LDF model (n = 20): 0.466

```

## Run every estimator once, on one freshly generated training sample.
## Returns a single ONE-ROW data frame -- one named column per estimator.
## Stacking many of these rows (one per replicate) is how "one experiment"
## becomes Table 3's Exp/SD/RMS summaries.
one_rep <- function(n, fit_fn, predict_fn, B=50, n_test=10000){

  ## All of the functions we just defined
  sample_data <- generate_data(n)
  model <- fit_fn(sample_data)
  preds <- predict_fn(model, sample_data)

  apparent_error <- mean(qloss_function(sample_data$y, preds))
  true_err <- true_error(model, predict_fn, n_test = n_test)
  cv1 <- loo_cv_error(sample_data, fit_fn, predict_fn)
  loo_boot_err <- loo_bootstrap_error(sample_data, fit_fn, predict_fn, B = B)
  err_632 <- estimator_632(apparent_error, loo_boot_err)
  gamhat <- gamma_hat(mean(sample_data$y), mean(preds))
  rhatprime <- R_hat_prime(loo_boot_err, apparent_error, gamhat)
  err_632_plus <- estimator_632_plus(err_632, loo_boot_err, gamhat, apparent_error, rhatprime)

  ## Put the results together! One row, one column per quantity.
  data.frame(
    true_err = true_err,
    apparent = apparent_error,
    cv1 = cv1,
    err1 = loo_boot_err,
    err_632 = err_632,
    err_632_plus = err_632_plus
  )
}

```

```

## Repeat one_rep() n_reps times and stack the one-row results into a
## single data frame: one row per replicate, one column per estimator.
montecarlo <- function(n_reps, n, fit_fn, predict_fn, B = 50, n_test = 10000){
  results_list <- vector("list", n_reps)

  for (r in seq_len(n_reps)) {
    results_list[[r]] <- one_rep(n, fit_fn, predict_fn, B = B, n_test = n_test)
  }

  do.call(rbind, results_list)
}

## Run the simulation for LDF, n = 20 (experiment #3), matching the
## paper's 200 Monte Carlo replications for this experiment
set.seed(17)
sim_results <- montecarlo(n_reps = 200, n = 20, fit_fn = ldf_fit, predict_fn = ldf_predict)

## Summarize: Exp (mean) and SD for each estimator column, and RMS
## measured against EACH replicate's own true_err (not the overall
## average true error) -- matching how Table 3 defines RMS
summary_table <- data.frame(
  estimator = names(sim_results),
  Exp = sapply(sim_results, mean),
  SD = sapply(sim_results, sd),
  RMS = sapply(sim_results, function(col) sqrt(mean((col - sim_results$true_err)^2)))
)
rownames(summary_table) <- NULL
summary_table

```

	estimator	Exp	SD	RMS
1	true_err	0.3545585	0.05683561	0.0000000
2	apparent	0.2507500	0.09208330	0.1412507
3	cv1	0.3462500	0.11935100	0.1187273
4	err1	0.3733764	0.10796744	0.1065094
5	err_632	0.3282499	0.09860228	0.1012579
6	err_632_plus	0.3571477	0.18377678	0.1817677

The table above replicates experiment 3 from Table 3 in the paper.

This demonstrates that the .362 estimator has the best overall performance for the LDF Classifier for $n = 20$, $p = 2$.

Sources:

- <https://www.geeksforgeeks.org/machine-learning/bias-vs-variance-in-machine-learning/>
- <https://www.geeksforgeeks.org/machine-learning/parametric-vs-non-parametric-models-in-machine-learning/>
- <https://www.statology.org/prediction-error-statistics/>
- Claude Sonnet 5

Part II

02: Visualizing the Model

4 In class exploration and discussion

Review of Master's Content: <https://tgstewart.cloud/ipm/visualizing-the-model.html>

Partial Effects Plot

- Hold all but one of the features constant, and plot the relationship between the variable feature and the outcome variable (ex. x_1 , x_3 are held constant, x_2 varies, y is the outcome. plot y vs x_2)

Individual Conditional Expectation (ICE) plot

- Use some information about the distribution of the features to investigate the relationship between a separate feature and the outcome variable
- vary x_1 and x_3 based on their distributions. vary x_2 in the same way it was before for the PE plot. For each iteration of the variation in x_1 and x_3 , plot y vs x_2 , and investigate the relationship.

Partial Dependence Plot (PDP)

- the average of each line on the ICE plot
- a marginal estimate of the PE plot

Accumulated Local Effect (ALE) plots

- needed when features are NOT independent (ex. BMI and weight are not independent, but could both be predictors in a model)
- uses partial derivatives to figure out the direction of the slope at different locations
- main take home message is that correlated predictors require special visualization techniques to tease apart the relationship that exists between the predictors and outcome

5 Partial Dependence Plots (PDP)

5.1 Student Exploration

Resources

1. [Elements of Statistical Learning \(ESL\) - Section 10.13.2](#)
2. [Molnar 2022 - Section 19](#)

5.2 Motivating Idea for the PDP

The Partial Effects plot and Independent Conditional Expectation (ICE) plot help with visualizing a model when we don't know the inner workings, specifically the relationships between features and outcomes. Partial Effects plots are used to investigate a single feature while holding all other features constant. ICE plots investigate a single feature while allowing the other features to vary across every individual value in the dataset. You can see how ICE plots may provide more information than the Partial Effects plot, but also provide too many lines on a given plot to be useful.

Enter the Partial Dependence Plot (PDP). We'll take the average line across all of the ICE plot lines, to investigate the average relationship of a feature to the predicted outcome.

Partial Dependence Plot (PDP)

A partial dependence plot shows the marginal effect one or two features have on the predicted outcome of an ML model. It can show whether the relationship between target and feature is linear, monotonic, or more complex.

The partial dependence function is defined as

$$\hat{f}_S(\mathbf{x}_S) = \mathbb{E}_{\mathbf{x}_C}[\hat{f}_S(\mathbf{x}_S, X_C)]$$

- $\mathbf{x}_S$ are the features which are plotted on the x axis of the PDP. Typically only one feature is in the set S . This is the feature we are investigating the effect of on the prediction.
- X_C are the other features used in the machine learning model $\hat{f}$
- If the feature vectors $\mathbf{x}_S$ and $\mathbf{x}_C$ are combined, they make up the total data $\mathbf{X}$

- The partial function $\hat{f}_S$ is estimated by calculating averages in the training data (aka. Monte Carlo):

$$\hat{f}_S(\mathbf{x}_S) = (1/n) \sum_{i=1}^n \hat{f}(\mathbf{x}_S, \mathbf{x}_C^{(i)})$$

- This step is equivalent to averaging all of the ICE curves of the dataset. For each of the points along the x axis for $\mathbf{x}_S$, you take the mean across the vertical direction ($\mathbf{x}_C^{(i)}$).
- So in this equation,
 - n is the number of instances in the dataset.
 - $\mathbf{x}_C^{(i)}$ are actual feature values from the dataset for features we are not interested in

5.3 Exploration

Let's explore how to create and interpret the PDP.

First, we'll generate some data from the following distribution.

$$E[Y | X] = \sqrt{5} \text{sigmoid}(X_1 + X_3) + \sqrt{5} \text{sigmoid}(X_2) X_3$$

$$V[Y | X] = 1$$

where $X_1, X_2 \sim N(0, 1)$

$$X_3 \sim \text{Bernoulli}(p = 0.4)$$

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

```
sigmoid <- function(x) 1 / (1 + exp(-x))

generate_data <- function(n) {
  X1 <- rnorm(n, mean = 0, sd = 1)
  X2 <- rnorm(n, mean = 0, sd = 1)
  X3 <- rbinom(n, size = 1, prob = 0.4)
  mu <- sqrt(5) * sigmoid(X1 + X3) + sqrt(5) * sigmoid(X2) * X3
  Y <- mu + rnorm(n, mean = 0, sd = 1)
  data.frame(Y, X1, X2, X3)
}

set.seed(17)
df <- generate_data(1000)
#head(df)
```

Now we'll fit some models on the data. First, a random forest.

```
library(randomForest)

train_data <- generate_data(1000)
test_data <- generate_data(1000)

model_formula <- Y ~ X1 + X2 + X3
rf <- randomForest(model_formula, data = train_data, ntree = 500)

preds <- predict(rf, newdata = test_data)
```

We have a fitted model that can generate predictions (now our Y values or outcomes), and we want to investigate the effect of say X_2 on Y .

Let's walk through building the PDP to see this.

First, here's what our data looks like

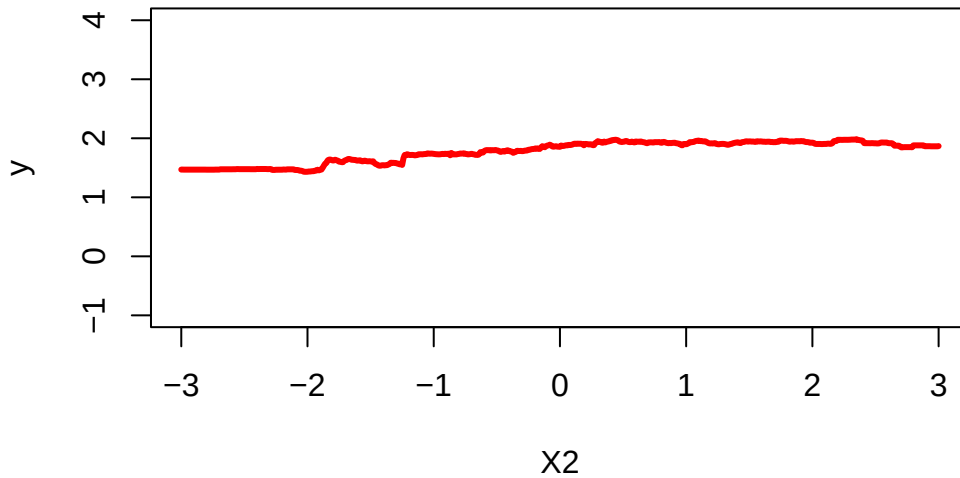
```
df <- generate_data(1000) # generate some data
preds <- predict(rf, newdata = df) # use the RF model to create predicted outcomes
df$preds <- preds # save predicted values to df
head(df)
```

	Y	X1	X2	X3	preds
1	2.1390719	0.05183928	-0.8736403	1	2.6070967
2	5.5903463	1.21341246	1.2258274	1	3.3018672
3	1.4677137	-0.98338726	-0.1526766	0	0.7781615
4	-0.7025197	-0.39636582	-0.8310759	0	1.1085454
5	2.1489275	-0.04981333	0.7814832	1	2.9208151
6	-1.1496933	-1.44647693	-1.2100006	0	0.7678897

Then, we'll want to visualize the PDP plot. Recall that we'll have our Predictions on the y-axis, X_2 on the x-axis. X_1 and X_3 vary across all of their real values in the dataset, and we get the average line to put on the plot.

```
# Create a partial effect plot for each draw, store it in O
O <- list()
for(i in 1:nrow(df)){
  R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
  O[[i]] <- predict(rf, newdata = R)
}

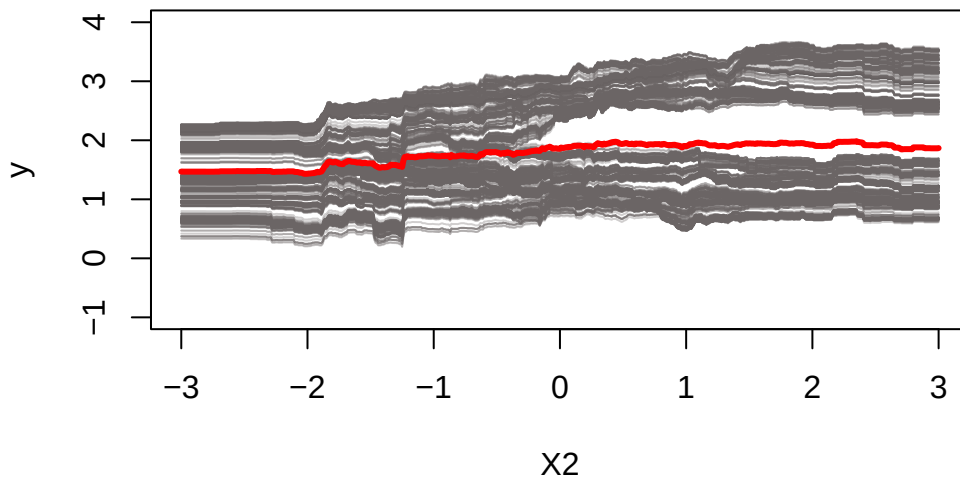
P <- do.call("cbind", O) |> rowMeans()
plot(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```



To contextualize this, it can be helpful to see the ICE plots with the PDP plot.

```
# Partial effect plot for the first random x
R <- expand.grid(X1 = df[1,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[1,"X3"])
plot(R[,2], predict(rf, newdata = R), xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4))

# Overlay the plot for all the xs
for(i in 2:nrow(df)){
  R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
  lines(R[,2], predict(rf, newdata = R), xlab = "X2", ylab = "y", type = "l", col = "#6b6595")
}
lines(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```



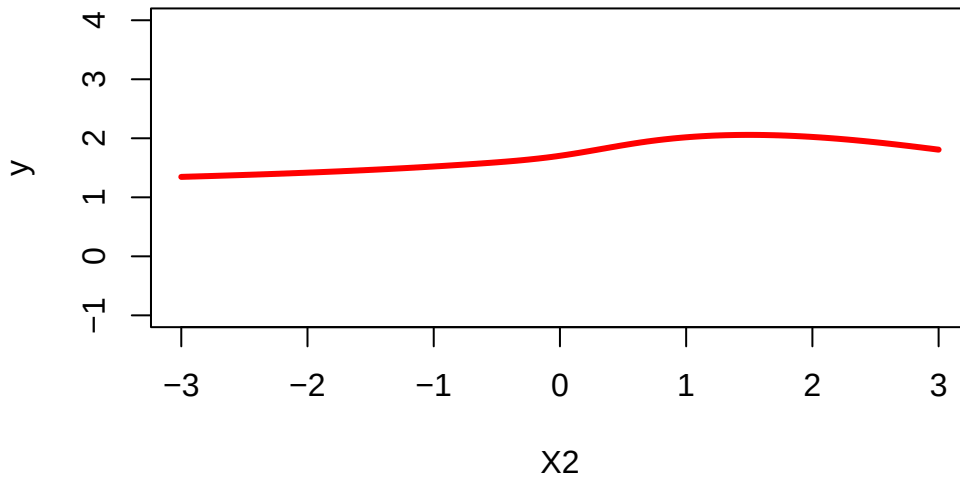
Next, let's repeat for the linear regression with splines.

```
library(splines)

lm_spline <- lm(Y ~ ns(X1, df = 4) + ns(X2, df = 4) * X3, data = df)

O <- list()
for(i in 1:nrow(df)){
  R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
  O[[i]] <- predict(lm_spline, newdata = R)
}

P <- do.call("cbind", O) |> rowMeans()
plot(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```



```
# Partial effect plot for the first random x
R <- expand.grid(X1 = df[1,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[1,"X3"])
plot(R[,2], predict(lm_spline, newdata = R), xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4))

# Overlay the plot for all the xs
for(i in 2:nrow(df)){
  R <- expand.grid(X1 = df[i,"X1"], X2 = seq(-3,3,by=0.01), X3 = df[i,"X3"])
  lines(R[,2], predict(lm_spline, newdata = R), xlab = "X2", ylab = "y", type = "l", col = "red")
}
lines(R[,2], P, xlab = "X2", ylab = "y", type = "l", ylim = c(-1,4), lwd = 3, col = "red")
```

